Web 2 – Séance 13

Collection comme état

← Toutes les séances

Le problème

Depuis la séance 10, les recettes viennent d'un module de données : un tableau **constant**.

- Le formulaire de la séance 12 crée une recette… qui n'apparaît nulle part
- Impossible de supprimer une recette de la liste

Pour que la liste change à l'écran, elle doit devenir un **état** :

```
// "as" renomme l'import : le nom recipes reste libre pour l'état
import { recipes as initialRecipes } from "./data/recipes";

const App = () => {
  const [recipes, setRecipes] = useState<Recipe[]>(initialRecipes);
  // …
};
```

Mais un tableau ou un objet dans l'état se manipule avec précaution.

Valeurs et références

- En JS/TS, une variable de type objet ou tableau ne contient pas l'objet en lui-même
- Elle contient à la place une **référence** vers l'objet en mémoire.
- `===` sur des objets compare les **références**, pas le contenu
- Le spread operator crée un nouvel objet ou tableau : une nouvelle référence

Comment React détecte un changement

Quand on appelle le setter fourni par `useState`, React compare l'**ancienne** et la **nouvelle** valeur avec `Object.is` (équivalent de `===`).

- Valeurs identiques → rien n'a changé, pas de nouveau rendu
- Valeurs différentes → nouveau rendu

```
const [recipes, setRecipes] = useState<Recipe[]>(initialRecipes);

// ❌ Mutation : même référence, React ne voit aucun changement
const addRecipe = (recipe: Recipe) => {
  recipes.push(recipe);
  setRecipes(recipes);
};

// ✅ Nouveau tableau : nouvelle référence, React refait le rendu
const addRecipe = (recipe: Recipe) => {
  setRecipes([...recipes, recipe]);
};
```

Règle : l'état est **immuable**. On ne le modifie jamais, on le **remplace** par une nouvelle valeur.

Ajouter un élément

```
// À la fin
setRecipes([ ... recipes, newRecipe]);

// Au début
setRecipes([newRecipe,  ... recipes]);

// Plusieurs éléments
setRecipes([ ... recipes,  ... otherRecipes]);

// Forme fonctionnelle (séance 11) si la mise à jour dépend de l'état précédent
setRecipes((prev) ⇒ [ ... prev, newRecipe]);
```

`[ ... recipes, newRecipe]` crée un nouveau tableau contenant les **mêmes objets** recette, plus le nouveau : c'est une copie superficielle (*shallow copy*), rapide même pour une grande liste.

Supprimer un élément

`filter` retourne un **nouveau tableau** avec les éléments qui passent le test.

```
const deleteRecipe = (id: number) => {
  setRecipes(recipes.filter((recipe) => recipe.id !== id));
};
```

- On garde toutes les recettes **sauf** celle dont l'id correspond
- Le tableau d'origine n'est pas modifié

Modifier un élément

`map` retourne un nouveau tableau ; on remplace l'élément concerné par une **copie modifiée**.

```
const renameRecipe = (id: number, title: string) => {
  setRecipes(
    recipes.map((recipe) =>
      recipe.id === id
        ? { ...recipe, title }   // nouvel objet pour la recette modifiée
        : recipe                 // même objet pour les autres
    )
  );
};
```

⚠️ Modifier l'objet recette lui-même est aussi une mutation :

```
// ❌ Le tableau est nouveau, mais l'objet recette est muté
setRecipes(recipes.map((r) => { if (r.id === id) r.title = title; return r; }));
```

Méthodes qui mutent un tableau

Méthode qui **modifie** le tableau	Alternative qui **crée** un nouveau tableau
`push(x)`	`[...arr, x]`
`unshift(x)`	`[x, ...arr]`
`splice(i, 1)`	`arr.filter((_, index) ⇒ index ≠= i)`
`arr[i] = x`	`arr.map((item, index) ⇒ (index === i ? x : item))`
`sort()`	`arr.toSorted()` ou `[...arr].sort()`
`reverse()`	`arr.toReversed()` ou `[...arr].reverse()`

Générer un identifiant

Chaque élément d'une liste a besoin d'un identifiant unique (pour la `key` notamment).

```
// Plus grand id existant + 1
const nextId = Math.max( ... recipes.map((r) ⇒ r.id)) + 1; // 4, si les ids existants sont 1, 2 et 3

// Nombre basé sur l'heure, en millisecondes
const timeId = Date.now();          // 1790000000000

// Identifiant aléatoire standard (UUID), de type string
const uuid = crypto.randomUUID();   // "550e8400-e29b-41d4-a716-446655440000"
```

- `Math.max( ... ) + 1` est simple, mais réutilise l'id d'une recette supprimée si c'était la plus grande, et donne `-Infinity` sur une liste vide
- `Date.now()` suffit si on ne crée pas plusieurs recettes dans la même milliseconde
- `crypto.randomUUID()` est plus sûr, mais impose un `id: string`
- Ce sont des identifiants **temporaires** : en Partie 4, c'est le backend qui attribuera l'id

Partage de l’état

- Un état est **local** à un composant : il n’est pas accessible aux autres composants
- Séance prochaine : comment l’état peut être **partagé** entre plusieurs composants, avec un parent qui le possède et le passe en prop aux enfants
- Pour cette séance, on va regrouper tout l’affichage qui a besoin de l’état dans un seul composant

Tableau d'objets dans un formulaire

Les ingrédients d'une recette forment une liste de longueur variable : un état tableau dans le formulaire.

```
interface IngredientInput {
  key: number;       // identifiant local, pour la key React
  name: string;
  quantity: string;  // chaîne, comme les autres champs numériques
  unit: string;
}
type IngredientField = "name" | "quantity" | "unit"; // champs modifiables

const [ingredients, setIngredients] = useState<IngredientInput[]>([]);
const addIngredient = () =>
  setIngredients([ ...ingredients, { key: Date.now(), name: "", quantity: "", unit: "g" }]);
const removeIngredient = (key: number) =>
  setIngredients(ingredients.filter((ingredient) => ingredient.key !== key));

const updateIngredient = (key: number, field: IngredientField, value: string) =>
  setIngredients(
    ingredients.map((ingredient) =>
      ingredient.key === key ? { ...ingredient, [field]: value } : ingredient
    )
  );
```

`IngredientField` : type union de chaînes littérales, seuls ces trois noms sont acceptés pour `field`.

Afficher la liste de champs

```
{ingredients.map((ingredient) => (
  <Stack key={ingredient.key} direction="row" spacing={1}>
    <TextField
      label="Ingrédient"
      value={ingredient.name}
      onChange={(e) => updateIngredient(ingredient.key, "name", e.target.value)}
    />
    <TextField
      label="Quantité" type="number"
      value={ingredient.quantity}
      onChange={(e) => updateIngredient(ingredient.key, "quantity", e.target.value)}
    />
    <TextField
      label="Unité"
      value={ingredient.unit}
      onChange={(e) => updateIngredient(ingredient.key, "unit", e.target.value)}
    />
    <IconButton onClick={() => removeIngredient(ingredient.key)} aria-label="Retirer">
      <DeleteIcon />
    </IconButton>
  </Stack>
))}
<Button onClick={addIngredient}>Ajouter un ingrédient</Button>
```

Récapitulatif Séance 13

- **Collection comme état** – `useState<Recipe[]>(initialRecipes)`
- **Références** – Une variable objet contient une référence ; `===` compare les références
- **Détection** – React compare ancienne et nouvelle valeur avec `Object.is`
- **Immuabilité** – Ne jamais modifier l'état, le remplacer par une nouvelle valeur
- **Ajouter** – `[ ...arr, x]`
- **Supprimer** – `arr.filter( ... )`
- **Modifier** – `arr.map( ... )` avec `{ ...item, prop }` pour l'élément concerné
- **Identifiants** – `Math.max( ... ) + 1`, `Date.now()` ou `crypto.randomUUID()`, temporaires jusqu'à la Partie 4

Prochaine séance : Séance 14 – État partagé

Exercice filé S13

Tout se passe dans `RecipeForm` : les ingrédients et les étapes sont des **états locaux** du formulaire.

1. Ajoutez un état `ingredients` (tableau de `IngredientInput`) : un champ nom, quantité et unité par ingrédient, un bouton pour retirer chaque ingrédient, et un bouton « Ajouter un ingrédient »
2. Ajoutez de la même façon un état `steps` : un champ texte multiligne par étape (chaque étape a aussi besoin d'un identifiant local pour sa `key`), avec des boutons pour ajouter et retirer une étape
3. Validez : au moins un ingrédient avec un nom et une quantité positive, au moins une étape non vide ; les erreurs s'affichent après la première soumission, comme en séance 12
4. À la soumission, l'objet affiché dans la console contient `ingredients` (quantités converties en nombres, sans la propriété `key`) et `steps` (un simple tableau de chaînes)
5. Après la soumission, videz tout le formulaire, ingrédients et étapes compris
6. **Optionnel** : ajoutez à chaque étape des boutons pour la monter ou la descendre dans la liste, sans muter le tableau

Exercice complémentaire EC07

1. Créez un projet `EC07` avec Vite + React + TypeScript
2. Commencez par afficher un catalogue de 5 produits statiques de votre choix,
   - Chaque produit affiche un id, un nom, un prix et un bouton « Ajouter au panier »
3. Le panier est une variable d'état contenant une collection d'objets de type `{ productId, quantity }`
   - Quand on clique sur « Ajouter au panier » pour un produit non présent, un nouvel objet est ajouté à la collection avec une quantité de 1
   - Quand on clique sur « Ajouter au panier » pour un produit déjà présent, sa quantité augmente de 1
4. Affichez en dessous du catalogue la liste des produits dans le panier
   - Chaque ligne du panier affiche le nom du produit, sa quantité, son prix total (prix * quantité) et trois boutons : « − », « + » et « Retirer »
   - Quand on clique sur « − », la quantité diminue de 1 ; si elle atteint 0, la ligne disparaît du panier
   - Quand on clique sur « + », la quantité augmente de 1
   - Quand on clique sur « Retirer », la ligne disparaît du panier
5. Affichez le total du panier (somme des prix totaux de chaque ligne) en dessous de la liste du panier